

Received 17 April 2025, accepted 7 May 2025, date of publication 9 May 2025, date of current version 16 May 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3568702

 SURVEY

Addressing Activation Outliers in LLMs: A Systematic Review of Post-Training Quantization Techniques

PATRIK CZAKÓ¹, (Student Member, IEEE), **GÁBOR KERTÉSZ**^{2,3}, (Senior Member, IEEE),
AND SÁNDOR SZÉNÁSI^{2,4}, (Member, IEEE)

¹Doctoral School of Applied Informatics and Applied Mathematics, Óbuda University, 1034 Budapest, Hungary

²John von Neumann Faculty of Informatics, Óbuda University, 1034 Budapest, Hungary

³Laboratory of Parallel and Distributed Systems, Institute for Computer Science and Control (SZTAKI), Hungarian Research Network (HUN-REN), 1111 Budapest, Hungary

⁴Faculty of Economics and Informatics, J. Selye University, 945 01 Komárno, Slovakia

Corresponding author: Patrik Czako (czako.patrik@stud.uni-obuda.hu)

ABSTRACT Large Language Models (LLMs) have transformed natural language processing, yet their deployment remains challenging due to substantial computational, memory, and energy demands. Post-training quantization has emerged as a key strategy for enabling efficient inference, particularly in resource-constrained settings. This systematic review focuses on weight-activation quantization, with a unique emphasis on the emergent outlier phenomenon in LLM activations. This work evaluates recent techniques that mitigate activation outliers and improve quantization efficiency, distinguishing itself from prior reviews. Using the PRISMA methodology, we examine 52 recent studies to uncover key trends and evaluate the effectiveness of different approaches. By synthesizing insights from these works, this review presents a diverse set of techniques and their implications for activation quantization, laying the groundwork for future research and practical advancements in LLM deployment.

INDEX TERMS Activation outliers, efficient inference, large language models (LLMs), model compression, quantization.

I. INTRODUCTION

The rapid advancement of artificial intelligence has been characterized by the exponential scaling of Large Language Models (LLMs) [1]. These models, now reaching hundreds of billions of parameters, far surpass their predecessors in scale [2] and have redefined benchmarks in natural language understanding and generation. This scaling trend has been driven by the observation that larger models tend to achieve superior performance across diverse tasks such as reasoning, translation, and summarization. However, the benefits of these large-scale models are accompanied by considerable challenges, including increased computational requirements, memory demands, and energy consumption.

The associate editor coordinating the review of this manuscript and approving it for publication was Wei Ni.

Deploying these massive models in practical applications necessitates adapting them for resource-constrained environments. Model compression [3], [4] helps adapt large models by transforming them into smaller, more efficient forms while preserving accuracy and generalization. Fine-tuning or retraining these models remains prohibitively expensive for most researchers due to the immense computational overhead involved. Post-training techniques provide a pragmatic alternative by allowing for the creation of compressed models using minimal calibration data in a single compression step, avoiding the need for retraining and keeping computational costs manageable.

Pruning and quantization are two leading strategies in model compression. Pruning eliminates redundant parameters, creating a sparse model that requires fewer computations and reduces memory usage. Quantization, on the other hand,

lowers the precision of model parameters by converting them to lower-bit formats such as INT8 or INT4. While both methods are effective, quantization is superior in minimizing accuracy loss and is highly compatible with hardware accelerators designed for low-precision arithmetic, making it an optimal choice for efficient deployment [5]. As a result, quantization is a widely adopted approach for accelerating inference and can also be applied effectively during fine-tuning [6]. This review focuses specifically on Post-Training Quantization (PTQ) techniques.

Recent work on LLM quantization [7], [8], [9], [10], [11], [12], [13] has primarily concentrated on weight-only quantization, a well-explored area targeting the acceleration of memory-bound inference. In such methods, activations remain in full precision, while weights are reduced to lower bit-widths, including as low as 1 bit in the case of LLM binarization [12], [13]. However, this approach has limitations, particularly in compute-bound scenarios where it falls short in delivering optimal performance gains. Activation quantization emerges as a promising solution to this challenge, albeit with substantial difficulties. In contrast to prior reviews [3], [4], [14], [15], this work focuses on activation outliers in LLMs and evaluates recent mitigation techniques. This study seeks to address the following research questions:

- Q1: What are the latest advancements in post-training activation quantization techniques for LLMs?
- Q2: How can activation distributions be adjusted to improve quantization efficiency while preserving numerical equivalence during computations?
- Q3: What alternative methods exist to minimize quantization errors in outlier-heavy activation distributions?
- Q4: How do these techniques perform relative to each other in terms of accuracy preservation, compression ratios, and processing time, and how do they compare against weight-only quantization and other computationally intensive methods?

By exploring these questions, this paper aims to contribute to the growing body of research on LLM compression, providing insights into cutting-edge techniques and their practical applications. Section II highlights the importance and challenges associated with activation quantization, particularly in addressing compute-bound inefficiencies. Section III then outlines the methodology for selecting and evaluating relevant studies based on the PRISMA framework [16]. This is followed by a detailed evaluation of selected papers in Section IV, leading to the conclusions summarized in the final section.

II. BACKGROUND

A. QUANTIZATION BASICS

Quantization techniques [14], [15] utilize discrete low-bit values to approximate high-precision floating-point representations, aiming to reduce storage requirements and improve computational efficiency. Among these methods, integer quantization [17] is the most widely used due to its simplicity,

energy efficiency, and broad hardware compatibility. This approach maps the floating-point values of a tensor $\mathbf{X}$ onto a discrete set of points defined by a uniform grid. Given a bit-width b , the grid's step size Δ and zero point z is computed as [18]:

$$\Delta = \frac{\max(\mathbf{X}) - \min(\mathbf{X})}{2^b - 1}, \quad z = -\lfloor \frac{\min(\mathbf{X})}{\Delta} \rfloor. \quad (1)$$

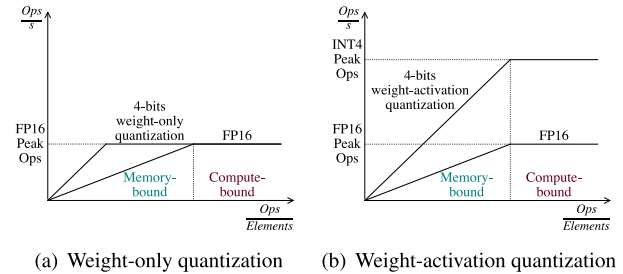


FIGURE 1. A roofline model of different quantization approaches with arithmetic intensity (Ops/Elements) on the horizontal axis and throughput (Ops/s) on the vertical axis.

Here, $\lfloor \cdot \rfloor$ represents the rounding function. Each element of $\mathbf{X}$ is then quantized to an integer value as follows:

$$\mathbf{X}_{INT} = \text{clamp}(\lfloor \frac{\mathbf{X}}{\Delta} \rfloor + z, 0, 2^b - 1). \quad (2)$$

Finally, the dequantized/reconstructed value is obtained by $Q(\mathbf{X}) = (\mathbf{X}_{INT} - z) \cdot \Delta$. The quantization parameters Δ and z can be determined through calibration data (*static quantization*) or calculated dynamically during runtime (*dynamic quantization*). While dynamic quantization introduces additional latency, it often reduces quantization error [4]. Furthermore, when prior knowledge indicates that the values are symmetrically distributed, the zero point can be set to zero (*symmetric quantization*) [4], [14], [18]. This adjustment simplifies computations, potentially leading to lower inference latency.

B. IMPORTANCE OF ACTIVATION QUANTIZATION

Transformer-based language models primarily consist of dense and self-attention layers, with matrix multiplication accounting for over 75% of inference time across various model sizes and input token lengths [19]. Inference dynamics can be broadly categorized as either compute-bound, where computational throughput is the limiting factor, or memory-bound, where data transfer rates between memory and processing cores create a bottleneck. Arithmetic intensity measures the balance between compute operations and memory operations, serving as a crucial metric for analyzing performance [20], [21], [22]. Higher arithmetic intensity values indicate that the compute throughput (Ops/s) is the primary bottleneck, characterizing a compute-bound scenario. Conversely, lower values suggest that performance is limited by memory bandwidth, indicating a memory-bound scenario. Fig. 1 illustrates the roofline model [23] for both

weight-only and weight-activation quantization. Leveraging this model, we analyze the phases of LLM inference through the lens of arithmetic intensity and underscore the critical role of weight-activation quantization.

1) PREFILL IS COMPUTE-BOUND

During the initial stage of LLM inference, all tokens in the system prompt are processed as a single batch and propagated through the network, involving intensive matrix-matrix multiplications. As illustrated in Fig. 1, weight-only quantization does not enhance throughput during this compute-intensive phase because the weights are dequantized to high-bit formats (e.g., FP16) prior to these multiplications. In contrast, weight-activation quantization allows computations to be performed using low-bit formats (e.g., INT4), significantly boosting peak performance at this stage.

2) DECODING IS MEMORY-BOUND

After processing the system prompt, the model generates output tokens sequentially in an auto-regressive manner, which results in low arithmetic intensity. Throughput improvements during this phase depend on reducing memory traffic. Weight-only quantization compresses weight matrices, thereby accelerating decoding. However, the Key-Value (KV) cache, whose size scales linearly with the sequence length and batch size, behaves similarly to weights but remains unquantized in weight-only methods. This limitation reduces the effectiveness of weight-only quantization in self-attention layers, highlighting the importance of activation quantization.

3) SERVING MAKES DECODING COMPUTE-BOUND

In LLM serving, batching multiple requests increases arithmetic intensity during decoding, improving GPU utilization. As a result, the decoding phase becomes limited by the maximum compute throughput of the hardware. In this setting, weight-only quantization does not reduce computational complexity sufficiently, limiting achievable throughput.

C. CHALLENGES OF ACTIVATION QUANTIZATION

Weight quantization is relatively straightforward due to the narrow range of weight values. However, this simplicity does not extend to activation quantization, which faces challenges from significant outliers in activation tensors, as illustrated in Fig. 2. Different linear layers in large language models demonstrate distinct outlier patterns.

First, prior studies [24], [25] have identified systematic outliers in attention projection layers and the up-projection layers of Feed Forward Network (FFN) blocks (Fig. 2(a)). These systematic outliers appear as a limited group of channels that contain values across all tokens that are several orders of magnitude higher than the typical range. Second, other investigations [26], [27], [28] have revealed another class of outliers, termed massive outliers or activation spikes, predominantly in the FFN down-projection layers (Fig. 2(b)).

Unlike systematic outliers, these activation spikes are token-specific and exhibit even greater magnitudes.

With a fixed bit-width b , activation outliers increase the quantization step size, which in turn raises the expected quantization error. In addition, the resulting quantization bins are unevenly populated, with the majority of values clustered in bins corresponding to the lowest absolute values. This imbalance leads to inefficient utilization of quantization bits. Low-magnitude values are frequently rounded to zero, resulting in underflow errors that degrade model performance.

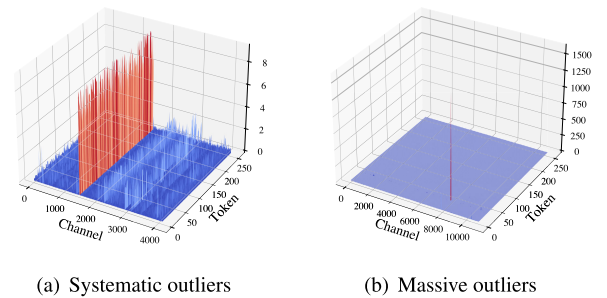


FIGURE 2. Magnitudes of input activations to the key projection (left) and down projection (right) modules in the second layer of LLaMA-7B.

III. METHODOLOGY

This section describes the method used to select and evaluate relevant research papers. We designed our methodology based on the PRISMA principles, which guide systematic reviews to ensure comprehensive and unbiased literature selection.

A. LITERATURE SEARCH AND IDENTIFICATION

In the identification step, we aimed to ensure comprehensive coverage of relevant papers. A literature search was conducted across three databases: Scopus, Web of Science (WoS), and ArXiv. Scopus and WoS served as the primary data sources due to their extensive journal coverage, while ArXiv was included to capture the latest preprints in the field.

To ensure comprehensive coverage and avoid missing relevant papers, we crafted a search query requiring the presence of the following terms in the title, abstract, or keywords: (“large language model” OR “LLM” OR “transformer” AND “language”)) AND “quantization”. Additionally, a publication year threshold was applied: 2022 for Scopus and WoS, and 2024 for ArXiv. The number of articles fitting these filters was 266 (Scopus), 88 (WoS), and 443 (ArXiv). Then 139 duplicates were removed, resulting in 658 unique papers.

B. ELIGIBILITY SCREENING

For our study, we defined the following eligibility criteria:

- Papers proposing quantization methods specifically designed for multi-billion scale LLMs.

- Studies on post-training quantization techniques emphasizing practical implementation with moderate computational requirements.
- Research addressing quantization of activations and/or the KV cache to tackle key challenges as described in Section II-C.

Based on these criteria, 474 articles were excluded during the abstract screening stage as they clearly fell outside the scope of this research. The full texts of the remaining 184 articles were retrieved directly from publishers or indirectly through platforms such as ScienceDirect, IEEE Xplore, Google Scholar, ResearchGate, or ArXiv. Three independent experts manually reviewed each paper against the eligibility criteria to ensure consistency and avoid bias. During this phase, an additional 135 articles were excluded. Furthermore, three new papers [22], [29], [30], not covered in the initial identification step, were added through citation searching. These were pre-2024 papers available only on ArXiv.

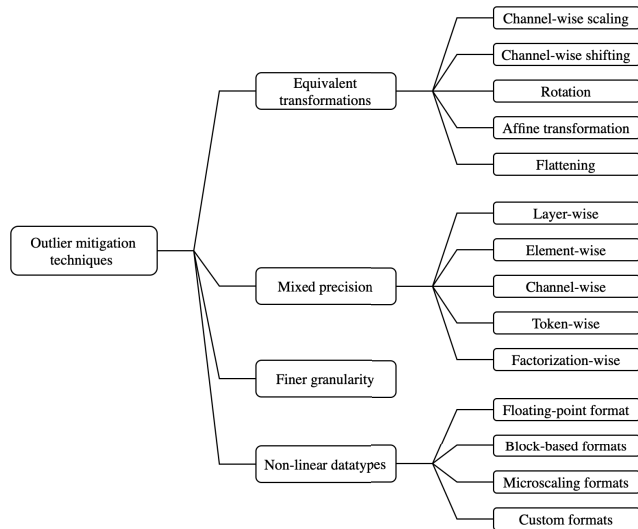


FIGURE 3. Taxonomy of techniques addressing the challenges of the outlier problem in LLM activation quantization.

Articles excluded during the screening step were classified into the following categories outside our objectives:

- Techniques designed for transformer-based models that are not primarily used for language processing (e.g., Vision Transformers) or for smaller-scale language models (e.g., BERT).
- Quantization-Aware Training (QAT) methods requiring extensive computational resources to produce compressed models.
- Approaches focusing solely on weight quantization while neglecting activations and KV cache quantization.
- Studies centered on hardware design rather than algorithmic innovations.
- Empirical or review articles that did not propose novel quantization techniques.

- Articles that were entirely unrelated to the research focus, likely included due to the broad and general nature of the search query.

C. ANALYSIS OF SELECTED PAPERS

The final set of selected papers consisted of 52 articles. These were systematically analyzed to evaluate the proposed quantization techniques, with a particular emphasis on outlier mitigation strategies. Detailed findings from this analysis are presented in the next section.

IV. EVALUATION

In this section, we evaluate the diverse strategies employed in the selected papers to address the challenges associated with LLM activation quantization, as introduced in Section II-C. We categorize the techniques into four groups: mathematically equivalent transformations to mitigate activation outliers, high-precision storage of outlier structures, smaller quantization groups to localize outlier impact, and non-linear datatypes for non-uniform grids. Fig. 3 presents a detailed taxonomy of these techniques, forming the structural basis for their evaluation in this section. Additionally, a comprehensive mapping matrix is provided in Table 1, linking the techniques to the corresponding papers. Following the conceptual overview, we conduct a quantitative analysis to compare their effectiveness in addressing the challenges of LLM activation quantization.

A. EQUIVALENT TRANSFORMATIONS

A widely adopted method employs linear transformations on input activations to reduce the impact of outliers while maintaining numerical equivalence.

1) CHANNEL-WISE SCALING

Outlier Suppression [31] observes that the output activations of the LayerNorm operation contain outliers in the same channels as the scaling parameter γ . This finding suggests that γ acts as an amplifier for these activation outliers. To address this, the authors propose the *Gamma Migration* method, which migrates γ into the subsequent residual branch and weight matrix.

SmoothQuant [25] extends this outlier suppression technique by introducing a per-channel scaling factor $\mathbf{s} \in \mathbb{R}^{c_{in}}$ (also referred to as the *smoothing factor*), where c_{in} represents the number of input channels. This scaling factor enables a numerically equivalent transformation of the linear layers, as expressed by the following formula:

$$\mathbf{Y} = \mathbf{X}\mathbf{W} = \underbrace{\left(\mathbf{X}\text{diag}(\mathbf{s})^{-1}\right)}_{\hat{\mathbf{X}}} \cdot \underbrace{\left(\text{diag}(\mathbf{s})\mathbf{W}\right)}_{\hat{\mathbf{W}}}. \quad (3)$$

In this formula $\mathbf{W}$, $\mathbf{X}$ and $\mathbf{Y}$ represent the weights, input, and output of the layer, respectively. Quantization is then performed on the transformed tensors $\hat{\mathbf{X}}$ and $\hat{\mathbf{W}}$. It is crucial to choose the smoothing factor carefully so that $\hat{\mathbf{X}} = \mathbf{X}\text{diag}(\mathbf{s})^{-1}$ is flattened token-wise, thereby

reducing quantization error. To determine an appropriate factor, SmoothQuant introduces the hyperparameter α to control the strength of the migration (i.e., the proportion of difficulty shifted to the weights) and uses the following equation:

$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}. \quad (4)$$

This per-channel scaling factor can be calibrated and fused into preceding layers offline. The paper demonstrates that W8A8 LLM quantization (utilizing fully INT8 matrix multiplication kernels) results in negligible accuracy loss, even when using per-tensor granularity.

FPTQ [29], however, argues that considering weights for the calibration of the smoothing factor is unnecessary. Instead, the researchers emphasize the importance of applying a non-linear lossless mapping to the activations, which can strongly suppress outlier channels while minimally affecting others. FPTQ proposes *Logarithmic Activation Equalization* (LAE) to compute the scaling factor as follows:

$$s_j = \max(|\mathbf{X}_j|) / \log_2(2 + \max(|\mathbf{X}_j|)). \quad (5)$$

CBQ [32] employs a coarse-to-fine outlier detection algorithm to identify a *truncated outlier set* $\mathbf{O}^*$. The maximum value in this set of outliers is then used to compute the scaling factor:

$$s_j = \sqrt{\max(|\mathbf{X}_j|) / \max(\mathbf{O}^*)}. \quad (6)$$

Unlike previous techniques, **QoQ** [21] also targets the smoothing of block intermediate activations (i.e., the outputs of the output projection layer in attention and the down projection layer in FFN). The authors observe that when optimizing the migration strength α in (4) for these activations, the resulting α is close to 0. This finding suggests that the smoothing factor $\mathbf{s}$ is primarily determined by the weights rather than the activations, which contradicts the observations in [25] and [29]. **I-LLM** [33] proposes *Fully-Smooth Block-Reconstruction* (FSBR), which extends smoothing to include the input activations of nonlinear operators, including gated activation functions.

In the case of KV cache quantization, researchers have found that channel-wise scaling can also shift quantization difficulty from Key ($\mathbf{K}$) to Query ($\mathbf{Q}$) vectors. QoQ introduces *SmoothAttention*, which applies equivalent scaling to the matrix multiplication in the attention operation:

$$\mathbf{Q}\mathbf{K}^\top = \underbrace{(\mathbf{Q}\text{diag}(\mathbf{s}))}_{\hat{\mathbf{Q}}} \cdot \underbrace{(\text{diag}(\mathbf{s})^{-1}\mathbf{K}^\top)}_{\hat{\mathbf{K}}^\top}. \quad (7)$$

In contrast to SmoothQuant, QoQ does not quantize Queries, allowing the focus to remain solely on the Keys when determining the SmoothAttention scaling factor as $s_j = \max(|\mathbf{K}_j|)^\alpha$. The authors impose the additional constraint $s_j = s_{j+\frac{c_{in}}{2}}$, to ensure that SmoothAttention is commutative with the *Rotary Positional Embedding* (RoPE).

QoQ calibrates the smoothing factors offline and fuses them into the weights of the preceding linear layers.

In [34], the researchers propose a similar approach to shift the quantization burden to the Query side using the computationally invariant transformation (7) for KV cache quantization. They compute the scaling factor $\mathbf{s}$ using the formula (4), as in SmoothQuant. Additionally, they perform calibration online during the prefill phase and address the impact of RoPE by employing a group size equal to half the attention head dimension.

Rather than relying on heuristics, several papers [44], [60], [65], [69], [72] aim to directly optimize the transformation based on the output. These approaches typically select the layer-wise reconstruction error as the optimization objective:

$$\underset{\mathbf{s}}{\text{argmin}} \left\| \mathbf{Q}(\hat{\mathbf{X}})\mathbf{Q}(\hat{\mathbf{W}}) - \mathbf{X}\mathbf{W} \right\|_2^2. \quad (8)$$

In [44], the researchers propose *Activation-Quantization-Aware Scaling* (AQAS), a method that optimizes the objective (8) using grid search to determine the appropriate scaling factor. Similarly, **QQQ** [69] introduces a technique called *Adaptive Smoothing*, which smooths only the channels with activation values exceeding a grid-searched outlier threshold.

Likewise, **ABQ-LLM** [65] aims to optimize (8) but also integrates parameters for *Learnable Weight Clipping* (LWC).

BiSup [72] employs *Parameter-Efficient Fine-Tuning* (PEFT) to adjust learnable parameters in order to minimize the layer-wise reconstruction error. The parameter space includes low-rank matrices similar to LoRA [73], as well as weight and activation clipping parameters and smoothing factors. Thanks to the use of fine-tuning, BiSup relaxes the numerical equivalence constraint of channel-wise scaling, allowing activations and weights to have distinct smoothing factors, $\mathbf{s}_1$ and $\mathbf{s}_2$, respectively, while ensuring the constraint $\text{diag}(\mathbf{s}_1) \cdot \text{diag}(\mathbf{s}_2) = \mathbb{I}$ is satisfied during parameter initialization.

2) CHANNEL-WISE SCALING AND SHIFTING

Although channel-wise smoothing effectively mitigates the impact of outlier concentration, it does not address channel asymmetry. To address this issue, some studies [39], [40], [55], [60], [64], [70] introduce an additional channel-wise shifting operation, resulting in the activation tensor $\hat{\mathbf{X}} = (\mathbf{X} - \mathbf{z})\text{diag}(\mathbf{s})^{-1}$, where $\mathbf{z} \in \mathbb{R}^{c_{in}}$ is the per-channel shifting parameter that eliminates the impact of asymmetric behavior caused by activation outliers. Accordingly, the equivalent transformation, considering the bias term $\mathbf{B}$, takes the following form:

$$\begin{aligned} \mathbf{Y} &= \mathbf{X}\mathbf{W} + \mathbf{B} \\ &= \underbrace{[(\mathbf{X} - \mathbf{z})\text{diag}(\mathbf{s})^{-1}]}_{\hat{\mathbf{X}}} \cdot \underbrace{[\text{diag}(\mathbf{s})\mathbf{W}]}_{\hat{\mathbf{W}}} + \underbrace{[\mathbf{B} + \mathbf{z}\mathbf{W}]}_{\hat{\mathbf{B}}}. \end{aligned} \quad (9)$$

Outlier Suppression+ [60] introduces a hyperparameter t , referred to as the *outlier threshold*, which is used to detect

TABLE 1. Matrix of quantization techniques used in selected papers, including usage counts.

Paper	Equivalent transformation				Mixed precision					Finer granularity	Non-linear datatype
	scaling	shifting	rotation	other	layer	element	channel	token	factorization		
[35]			✓				✓			✓	
[22]							✓				
[36]											
[27]								✓		✓	
[24]							✓				
[32]	✓										
[37]						✓					
[38]								✓		✓	
[39]	✓	✓									
[40]	✓	✓									
[41]	✓							✓		✓	
[42]						✓		✓			✓
[33]	✓										
[43]						✓		✓		✓	
[44]	✓										✓
[45]											✓
[29]	✓										
[28]	✓		✓								
[21]	✓		✓							✓	
[46]				✓							
[47]							✓			✓	
[48]									✓		
[49]											✓
[50]										✓	
[51]								✓		✓	
[52]			✓								
[53]	✓	✓	✓	✓							
[54]			✓								✓
[55]	✓	✓									
[56]								✓			
[57]	✓		✓	✓						✓	
[58]						✓					
[59]					✓					✓	
[31]	✓										
[60]	✓	✓									
[25]	✓										
[26]					✓			✓			
[34]	✓							✓		✓	
[61]										✓	
[62]										✓	
[63]	✓		✓							✓	
[30]										✓	
[64]	✓	✓								✓	
[65]	✓										
[66]											✓
[67]							✓				✓
[68]	✓			✓	✓						
[69]	✓										
[70]	✓	✓									
[20]							✓			✓	
[71]											✓
[72]	✓							✓			
Σ	23	7	8	4	3	4	6	10	1	18	8

outlier channels and constrain their values within $(-t, t)$. The outlier threshold is determined through grid search to minimize the layer-wise reconstruction error, with the parameters in (9) computed as follows:

$$z_j = \frac{\max(\mathbf{X}_j) + \min(\mathbf{X}_j)}{2},$$

$$s_j = \max\left(1, \frac{\max(\mathbf{X}_j - z_j)}{t}\right). \quad (10)$$

OmniQuant [55] introduces *Learnable Equivalent Transformation* (LET), where the channel-wise scaling and shifting factors in (9), as well as the attention scaling factors in (7), are jointly optimized using gradient descent to minimize the block-wise reconstruction error. Similar to ABQ-LLM [65], OmniQuant incorporates LWC, adding additional parameters to the optimization space.

While OmniQuant and ABQ-LLM initialize the scaling factor of LET using (4), following SmoothQuant [25], **LRQuant** [70] reports similar findings to FPTQ [29], namely that logarithmic equivalence is more effective at suppressing activation outliers. Consequently, LRQuant calculates the initial values using LAE, as formulated in (5), before proceeding with block-wise optimization. Additionally, LRQuant enhances the *Mean Squared Error* (MSE) loss with the *Negative Logarithm of Cosine Similarity* (NLC) to formulate the final optimization objective, which also accounts for directional similarity.

Other studies [39], [40], [64] build upon the quantization framework provided by OmniQuant, enhancing it with additional concepts. In [39], the authors introduce *Learnable Singular Value Increment* (LSI), which employs *Singular Value Decomposition* (SVD) to extract the singular values of the weights and introduces a new learnable parameter $\mathbf{I}'$ that is added to the singular values to adjust

the distribution of the original weights. However, LSI proves difficult to train and suffers from severe overfitting issues. To address these challenges, a follow-up study [40] proposes *Diagonal Expansion of Learnable Singular Values (DESV)*, a method that improves LSI by introducing additional learnable parameters, which are mapped to the super- and sub-diagonal elements of the singular value matrices.

WKVQuant [64] omits the quantization of temporary activations, focusing solely on weights and the KV cache. It introduces *Cross-block Reconstruction Regularization (CRR)* to address two key weaknesses of OmniQuant's block-wise optimization method. First, CRR incorporates multiple subsequent layers in the forward pass to achieve a closer approximation of the network's final output. Second, it replaces the MSE loss with *Mean Absolute Error (MAE)*, which reduces the error amplification caused by outliers. WKVQuant initializes the equivalent parameters as $s_j = \max(|\mathbf{X}_j|)$ and $z_j = \text{mean}(\mathbf{X}_j)$, respectively.

While the presented techniques achieve acceptable accuracy loss with channel-wise shifting and scaling, several notable shortcomings remain, particularly at lower precision. First, activation outliers are not eliminated but are partially migrated to the weights, increasing weight quantization errors. Second, the offline-calibrated smoothing scales often fail to represent the distribution of online data during inference, resulting in higher activation quantization errors. Third, the channel-wise scaling of massive outliers frequently introduces underflow errors during the quantization of normal values, which are scaled down near 0. Although the first issue can be mitigated by avoiding the migration of difficulty to weights, as implemented in **Runtime Smooth** [63], this approach requires additional techniques to minimize the computational overhead of online scaling. Solutions to the second and third problems, however, remain an open area of research.

3) ROTATION

In the context of weight-only LLM quantization, [7] introduced the concept of *incoherence processing*, which defines the weight matrix $\mathbf{W} \in \mathbb{R}^{n \times m}$ as μ -incoherent if $\max(|\mathbf{W}|) \leq \mu \|\mathbf{W}\|_2 / \sqrt{nm}$. High incoherence leads to greater quantization errors, as it indicates that the largest element in the matrix is an outlier relative to the average element's magnitude. The same concept can be applied to activation matrices, where incoherence serves as a measure of massive outliers. [7] demonstrated that incoherence can be reduced by multiplying the matrix on both sides by an orthogonal matrix. Building on this observation, **QuaRot** [35] introduces rotations for preprocessing activations prior to quantization, by multiplying the matrices with an orthogonal matrix $\mathbf{R}$ (a rotation matrix). This operation redistributes outlier values across all channels. Since $\mathbf{R}\mathbf{R}^\top = \mathbb{I}$ holds due to orthogonality, the following equivalent transformation can be

formulated:

$$\mathbf{Y} = \mathbf{X}\mathbf{W} = \underbrace{(\mathbf{X}\mathbf{R})}_{\hat{\mathbf{X}}} \cdot \underbrace{(\mathbf{R}^\top \mathbf{W})}_{\hat{\mathbf{W}}}. \quad (11)$$

Therefore, rotation provides a remedy for the third issue of channel-wise scaling by alleviating spike outliers. QuaRot also addresses the second problem since it is calibration-free, employing random Hadamard matrices to rotate weights and activations. Most of these rotation matrices can be absorbed into the weights of preceding layers; however, a small number of online Hadamard transformations must be inserted into the forward pass (e.g., after non-linear operations). Through these online rotations, QuaRot effectively reduces the magnitude of outliers in Keys and Values before quantizing the KV cache. This is achieved, in part, by applying random Hadamard transformations to the matrix multiplication $\mathbf{Q}\mathbf{K}^\top$ in attention, analogous to (7), where channel-wise scaling is employed for this purpose. In related works [21], [27], [54], [63], researchers incorporate rotation transformations into their methodologies based on the scheme introduced by QuaRot.

Similarly to the case of hand-crafted smoothing scales, the same question arises regarding random rotation matrices: *Can a lower quantization error be achieved by optimizing rotation matrices?* To address this question, **DuQuant** [28] proposes a greedy search algorithm to approximate the ideal rotation matrix. The algorithm runs for N steps, during which the values of the channel containing the largest absolute value are redistributed across all channels, while other channels are randomly rotated at each step k using the following rotation matrix:

$$\mathbf{R}^k = \mathbf{P}^k \tilde{\mathbf{R}} \tilde{\mathbf{Q}} \mathbf{P}^k, \quad \tilde{\mathbf{Q}} = \begin{bmatrix} 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{Q} \end{bmatrix}. \quad (12)$$

In this formula, $\mathbf{P}^k$ is a permutation matrix that swaps the first channel with the channel containing the largest absolute value, $\tilde{\mathbf{R}}$ is a rotation matrix whose first row is uniformly distributed to ensure outlier mitigation, and $\mathbf{Q}$ is a random orthogonal matrix. The rotation matrix is computed as $\mathbf{R} = \mathbf{R}^1 \mathbf{R}^2 \cdots \mathbf{R}^n$, where $n \in [1, N]$ is chosen to minimize the maximum absolute value in $\mathbf{X}\mathbf{R}$. To reduce computation and memory overhead, DuQuant approximates the rotation matrix in a block-wise manner, resulting in a block-diagonal structure. After this operation, different blocks may exhibit diverse ranges of outliers. To address this, DuQuant performs a zigzag permutation to balance the distribution of outliers across the blocks and then applies a second rotation. The final equivalent transformation is formulated as follows:

$$\mathbf{Y} = \mathbf{X}\mathbf{W} = \underbrace{[(\mathbf{X} \wedge) \mathbf{R}_1 \mathbf{P} \mathbf{R}_2]}_{\mathbf{G}} \cdot \underbrace{[\mathbf{R}_2^\top \mathbf{P}^\top \mathbf{R}_1^\top (\Lambda^{-1} \mathbf{W})]}_{\mathbf{G}^{-1}}, \quad (13)$$

where $\Lambda = \text{diag}(\mathbf{s})^{-1}$ is the smoothing matrix based on SmoothQuant [25], applied before the rotations, and $\mathbf{G}$ represents the final matrix of the equivalent transformation. This matrix can be calibrated offline and fused into preceding layers for most operations.

It is worth noting that permutation is a special case of rotation, where each row and column in its matrix contains exactly one entry of 1, with all other entries being 0. Consequently, permutation qualifies as a mathematically equivalent transformation; however, it does not inherently mitigate outliers. Despite this limitation, permutation remains a widely adopted technique in quantization, as it facilitates the grouping of channels with similar characteristics into finer-granularity groups [20], [21], [30], [38], [47], [63].

SpinQuant [52] surpasses previous approaches by directly optimizing offline rotation matrices using *CayleySGD* [74], an efficient optimization algorithm on the *Stiefel* manifold (i.e., the set of all orthonormal matrices). Consistent with QuaRot [35], SpinQuant also applies online random Hadamard transformations where necessary.

4) AFFINE TRANSFORMATION

AffineQuant [53] builds upon the previously discussed approaches by removing the constraint that the transformation matrix must be diagonal (channel-wise scaling) or orthogonal (rotation). Instead, it advocates for learnable equivalent affine transformations:

$$\mathbf{Y} = \mathbf{X}\mathbf{W} + \mathbf{B} = \underbrace{(\mathbf{X} - \mathbf{z})\mathbf{A}^{-1}}_{\hat{\mathbf{X}}} \cdot \underbrace{[\mathbf{A}\mathbf{W}]}_{\hat{\mathbf{W}}} + \underbrace{[\mathbf{B} + \mathbf{z}\mathbf{W}]}_{\hat{\mathbf{B}}}. \quad (14)$$

AffineQuant calibrates the equivalent parameters $\mathbf{z}$ and $\mathbf{A}$ to minimize the block-wise MSE loss and ensures the invertibility of $\mathbf{A}$ using a freezing mechanism during the optimization process, referred to as *Gradual Mask* (GM). According to the *Levy-Desplanques* theorem, strictly diagonally dominant matrices are invertible, a property guaranteed by GM. AffineQuant excludes the affine transformation between the two linear layers in the FFN module due to the optimization challenges posed by the large transformation matrix in high-dimensional spaces. Additionally, given the computational costs associated with the transformation matrix, AffineQuant does not support online transformations applied after non-linear operations, as seen in QuaRot [35] and SpinQuant [52], where Hadamard matrices can be reused across layers.

FlatQuant [57] proposes a similar technique, addressing the aforementioned limitations. It decomposes the affine transformation matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ using *Kronecker Decomposition* into $\mathbf{A} = \mathbf{A}_1 \otimes \mathbf{A}_2$, where $\mathbf{A}_1 \in \mathbb{R}^{n_1 \times n_1}$ and $\mathbf{A}_2 \in \mathbb{R}^{n_2 \times n_2}$ are smaller, invertible matrices, and $n = n_1 n_2$. To maintain training stability and efficiency, the authors leverage SVD and *Automatic Mixed Precision* (AMP) during the training process for FlatQuant.

5) FLATTENING

FlattenQuant [68] introduces a transformation technique called flattening, which serves a similar purpose to channel-wise scaling but avoids transferring quantization difficulty to the weights. Its primary objective is to achieve uniformly distributed values. The method identifies outlier channels by

detecting those whose maximum absolute value exceeds a truncation threshold t , and flattens the outlier channel j into $e_j = \lfloor \max(|\mathbf{X}_j|)/t \rfloor$ channels. Consequently, the algorithm introduces $e_j - 1$ new channels in both activation and weight matrices. For the weight matrix, values are repeated to ensure computational invariance.

This technique, however, incurs additional computational costs due to the increased number of channels in the matrix multiplication. **QLLM** [46] proposes a nearly identical transformation, termed channel disassembly. To offset the channel increase, QLLM applies adaptive reassembly, which averages input features using the following approximation:

$$x_i \mathbf{W}_{i,k} + x_j \mathbf{W}_{j,k} \approx \frac{x_i + x_j}{2} (\mathbf{W}_{i,k} + \mathbf{W}_{j,k}). \quad (15)$$

Therefore, this method does not maintain mathematical equivalence, but the extent of the introduced error can be controlled by the adaptively determined reassembly ratio θ for each layer.

B. MIXED PRECISION

An alternative approach involves allocating higher bit precision specifically for outlier structures, often retaining them in FP16. This strategy avoids modifying the input activations directly, while ensuring that critical values are represented with greater accuracy.

1) LAYER-WISE PRECISION

In this setting, different layers of the model operate at varying precision levels. **FlattenQuant** [68] quantizes weights and activations to 4 or 8 bits depending on the quantization sensitivity of a given layer. The sensitivity is evaluated by calculating the ratio of the *Kullback–Leibler* (KL) divergences between the original and quantized tensors at the two precision levels. **QFeM** [26] proposes avoiding the quantization of input activations in linear layers where significant quantization errors arise due to activation spikes. These spikes are identified using the max-median ratio as a metric. **AsymKV** [59] argues that quantization errors in the earlier layers are amplified as they propagate through deeper layers. To address this issue in the context of KV cache quantization, AsymKV adopts a higher precision (2 bits) for the initial layers, while applying 1-bit quantization to the remaining decoder layers. The quantization boundaries for Keys and Values are set asymmetrically to account for their differing quantization complexities.

2) ELEMENT-WISE PRECISION

Individual elements within tensors are stored at different precision levels, independent of their token, channel, or layer association. This fine-grained strategy is commonly applied to KV cache, utilizing the dense-and-sparse quantization technique, which has demonstrated considerable success in weight-only quantization [9], [10]. The core concept involves identifying individual outliers and storing them separately in a sparse matrix at half precision, while retaining the

normal values in a dense format with the outlier positions set to zero. The final result is then computed by performing two separate matrix multiplications and summing their outputs. **KVQuant** [42] and **GEAR** [43] implement per-channel outlier detection, with experiments reporting outlier ratios of 1% and 2%, respectively. KVQuant uniformly quantizes the dense matrices to 2, 3, or 4 bits, while GEAR adopts a differentiated approach, using 4 bits for system prompts (processed during the prefill phase) and 2 bits during auto-regressive decoding. Similarly, **QAQ** [37] sets the outlier ratio to 1% but treats the quantized Keys and Values as independent random variables. QAQ determines the bit allocation for dense matrices by minimizing the cache size under constraints on their standard deviation. **AlignedKV** [58] introduces a novel method for achieving element-wise precision, referred to as *Precision Alignment*. This approach stores the KV cache in FP16 format but applies mantissa truncation to the elements during data transfer from GPU memory to the L1 cache. Consequently, the average bit width per element is reduced to approximately 12, while preserving the performance of the original model.

3) CHANNEL-WISE PRECISION

Distinct precision levels are assigned to specific channels across all tokens, effectively mitigating the issue of systematic outliers. **LLM.int8()** [24] quantizes more than 99.9% of values to 8 bits, reserving 16 bits for the extreme outliers. This decomposition results in two separate matrix multiplications per layer, whose outputs are accumulated in FP16 and combined to produce the final result. Although this method achieves approximately a 50% reduction in memory compared to a 16-bit format, it can be challenging to implement efficiently on hardware accelerators, often leading to slower inference. To address this limitation, **QUIK** [22], **Atom** [20] and **FMPQ** [47] determine outlier channels during an offline calibration, and reorder activations and weights to ensure regular memory accesses for mixed-precision operations, employing 8 bits to store outliers and 4 bits for normal values. In [36], the authors introduce a three-tier mixed-precision quantization strategy that assigns high precision (16 bits) to magnitude-based outliers, intermediate precision (4/6/8 bits) to Hessian-based important channels, and low precision (1/2 bits) to the rest.

4) TOKEN-WISE PRECISION

Entire tokens (across all channels) are stored at mixed precision, with different tokens using different precision levels. This method enables adaptive precision allocation based on token importance or contextual needs. **ZipCache** [41], for example, quantizes the most salient tokens to 4 bits and the remaining tokens to 2 bits, using the normalized attention score as saliency metric. **MiKV** [34] employs an existing importance-based token eviction strategy [75] but, rather than removing less important tokens entirely, retains them at reduced precision (e.g., 2 bits). Other works emphasize

that recently generated tokens and attention sinks (i.e., the first tokens in a prompt) should also be preserved at higher precision. **SKVQ** [38] maintains a sliding window of the most recently generated tokens in full precision. Similarly, **KIVI** [51] and **GEAR** [43] employ a streaming buffer approach during decoding, retaining newly generated tokens at full precision. Both SKVQ and **KVQuant** [42] preserve the precision of attention sinks, while **BiSup** [72] entirely forgoes quantization of the system prompt. Recent techniques have observed that outlier tokens often occur in tokens with low semantic value (e.g., “\n”, “.”, etc.), with the largest scale typically allocated to the first instance of such tokens, while subsequent occurrences exhibit a more modest scale. Building on this observation, emerging methods integrate this effect with attention sinks by prefixing the system prompt with selected tokens that are retained at full precision. **QFeP** [26] constructs a prefix [**BOS**, **C**, **T**], where **BOS** is the *Beginning of Sequence* token, the candidate token **T** represents the activation spike at the linear layer with the highest max-median ratio, and the context token **C** designed to activate **T**. Similarly, **PrefixQuant** [27] identifies outlier tokens based on max-median ratio but adjusts the prefix size dynamically, setting it to the average number of outlier tokens across transformer blocks rather than fixing it at 3. The prefix is then composed of the highest-frequency outliers along with the **BOS** token. **CushionCache** [56] employs a greedy prefix search algorithm to iteratively select hard prompt tokens (tokens from the vocabulary) to initialize the prefix. It further enhances this strategy with *Quantization-Aware Prefix Tuning*, which freezes the model parameters and fine-tunes only the prefix.

5) FACTORIZATION-WISE PRECISION

The activation tensors are factorized into multiple sub-tensors. Each factor is assigned a distinct precision level, with higher precision allocated to the more critical components. **DecoQuant** [48] applies the *Matrix Product Operator* (MPO) [76] to decompose the original matrix into two local tensors: one significantly larger with a narrower value distribution than the original matrix, and the other smaller. This approach shifts the quantization complexity from the original matrix to the sub-tensors by applying low-bit quantization to the larger tensor while preserving high-precision representation for the smaller tensor. Experiments with other tensor decomposition techniques, such as SVD and QR, demonstrate that MPO achieves significantly lower quantization error while maintaining a higher compression ratio.

C. FINER GRANULARITY

As mentioned in Section II-C, the range of activation tensors can be very large, precluding their quantization as a single unit. Per-channel quantization would ideally address this issue due to the systematic outlier phenomenon. However, it remains impractical because it does not align well with

hardware-accelerated GEMM kernels. Consequently, most approaches employ per-token activation quantization to strike a balance between efficiency and performance.

The weight-only quantization technique GPTQ [77] has shown that separately quantizing small groups of weights can significantly enhance LLM quantization performance. Building on this insight, **ZeroQuant** [62] extends the idea to joint weight-activation quantization by adopting a finer-grained group-wise quantization for weights and per-token quantization for activations, while **ZeroQuant-V2** [61] explores the case of per-group activation quantization. **RPTQ** [30] clusters channels with similar value ranges and introduces channel reordering, ensuring low variation in quantization groups. **Atom** [20] and **COMET** [47] build on ZeroQuant-V2 and RPTQ, applying channel permutation and introducing supplementary techniques that further reduce the bit-width and accelerate the inference process.

In the case of fine-grained KV cache quantization, most works [20], [21], [27], [34], [35], [38], [57], [64] apply token-wise group quantization, typically using a group size that matches the dimensionality of the attention head (128 in most cases) to effectively leverage the memory-bound characteristics of self-attention. Token-wise quantization suffices for Values, but **KIVI** [51] shows that systematic outliers in specific channels make channel-wise quantization necessary for Keys. However, unlike token-wise quantization, the channel-wise option does not align well with the streaming setting. To address this, KIVI introduces a streaming buffer that gathers newly generated tokens and performs quantization whenever the buffer is full. **AsymKV** [59] and **GEAR** [43] adopt the same quantization scheme. Additionally, these techniques use smaller groups (32 or 64) to further reduce quantization errors.

Unlike the previously mentioned methods, **CrossQuant** [50] augments token-wise activation quantization by introducing additional channel-wise parameters $c_j = \max(|\mathbf{X}_j|)$ and integrates these scaling factors into its quantization scheme. **ZipCache** [41] proposes a similar technique to quantize Values, referred to as channel-separable token-wise quantization. It uses the square root of the channel-wise scaling factor $\mathbf{c}$ mentioned above.

D. NON-LINEAR DATATYPES

Although integer quantization remains the most widely used method because of its computational efficiency and hardware compatibility [17], it struggles to accurately represent the activation distributions in LLMs, particularly at lower bit widths. This section introduces approaches that mitigate these limitations by leveraging non-linear datatypes.

1) FLOATING-POINT FORMAT

In contrast to integer quantization, floating-point (FP) quantization offers greater flexibility, making it more effective in handling long-tail and bell-shaped distributions. Moreover, floating-point arithmetic has become increasingly favored

on modern hardware platforms, such as the NVIDIA H100. However, the performance of FP quantization is highly sensitive to the selection of exponent bits and clipping ranges, presenting a key implementation challenge. To address this, **LLM-FP4** [49] introduces a search algorithm that optimizes the data format and exponent biases by minimizing layer-wise reconstruction error. Furthermore, to resolve the systematic outlier problem, LLM-FP4 proposes a novel *pre-shifted exponent bias* technique. This method decomposes the real-valued exponent bias into a tensor-wise real-valued component and a channel-wise integer component. The channel-wise component can then be re-parameterized as the exponent bias of the corresponding weights, achieving numerical equivalence to the channel-wise scaling technique described in Section IV-A1.

2) BLOCK-BASED FORMATS

The study in [66] suggests that in addition to employing non-linear data types, sensitive layers should undergo fine-grained quantization. The researchers conducted experiments with various block-based data formats, including *Block Minifloat* (BM), *Block Logarithm* (BL), and *Block Floating Point* (BFP). Block-based quantization reduces the memory footprint by sharing common digits among elements within each block, effectively combining the precision of floating-point numbers with the efficiency of integer formats. Building on the Block Floating Point format, **BiE** [71] introduces a bi-shared exponent approach, where one exponent handles normal values and another handles outlier values. Unlike the mixed-precision methods described in Section IV-B, BiE employs a unified format that shares arithmetic units, encoders, and decoders, ensuring higher hardware efficiency.

3) MICROSCALING FORMATS

To improve low-precision LLM inference, the *microscaling format* (MX) [78] has been developed from BFP by introducing a shared scale across a block of reduced-precision elements, effectively mitigating quantization errors caused by limited dynamic range. While MX efficiently reduces activation outliers without requiring costly calibration, it introduces asymmetry within blocks. To overcome this limitation, **AMXFP4** [45] extends the MX format by incorporating an asymmetric numerical representation. It combines FP4 elements with shared asymmetric FP8 scales, enabling low-overhead computations and significantly reducing quantization errors. Compared to MXFP4, AMXFP4-FP8 achieves over 8x efficiency with only a 10% increase in hardware cost.

4) CUSTOM FORMATS

In [44], the researchers introduce the hybrid data type dINT, which combines integer and denormal representations to address the underflow problem, where small values are rounded to zero. While dINT was initially designed for weight quantization, the authors also explore its application in quantizing Values. In weight-only quantization,

clustering-based non-uniform scalar/vector quantization techniques are widely adopted [8], [10]. However, these techniques cannot be practically employed on-the-fly for activations due to the need for an expensive search over a codebook. In the decoding phase, where the KV cache acts as weights, the loading is memory-bound, and the dequantization overhead from non-uniform quantization methods remains negligible. **KVQuant** [42] proposes the per-tensor sensitivity-weighted non-uniform data type nuqX for KV cache quantization. This datatype is derived offline using calibration data and the K-means algorithm. It is subsequently rescaled per-channel or per-token to accurately capture the distributions of Keys and Values. Based on the observation in [67] that the joint entropy of Key/Value channels increases more slowly than the sum of their marginal entropies, the authors introduce a vector quantization technique called *Coupled Quantization (CQ)*. CQ divides Keys and Values into equally sized groups of contiguous channels, which are coupled and jointly quantized, sharing a single quantization code. Similar to KVQuant, CQ learns the centroids offline using a calibration dataset by leveraging the K-means algorithm weighted by second-order information. Extending the concept of vector quantization to activations, [54] introduces *Pyramid Vector Quantization (PVQ)* for Large Language Models. Unlike CQ, which views vectors in the Euclidean space, PVQ projects points onto the unit sphere using an implicit codebook. This codebook does not need to be instantiated in memory to perform quantization and can encode and decode vectors without requiring a search over the codebook.

TABLE 2. Comparison of channel-wise scaling techniques based on WikiText-2 perplexity for LLaMA-1 models.

Method	Scaling factor	Bits	Quantization granularity		Perplexity (↓)			
			W	A	7B	13B	30B	65B
Baseline	-	FP16	-	-	5.68	5.09	4.10	3.53
SmoothQuant [25]	(4)	W8A8 [*]	per-tensor	per-tensor	5.73	5.13	4.23	-
		W6A6 [†]	per-channel	per-token	6.03	5.42	4.55	3.88
		W4A4 [‡]	per-channel	per-token	25.25	40.05	192.40	275.53
FPTQ [29]	(5)	W6A6 [*]	per-group	mixed	6.04	5.43	4.57	-
		W4A4 [‡]	per-group	mixed	135.72	151.02	388.53	-
		W4A8	per-channel	per-tensor	-	-	4.32	3.84
CBQ [32]	(6)	W4A4	per-channel	per-token	-	-	7.96	5.89
AQAS [44]	(8)	W4A8	per-channel	per-token	6.48	5.67	4.72	4.13
QQQ [69]	(8)	W4A8	per-channel	per-token	6.19	5.43	4.61	-
		W4A4	per-group	per-token	5.87	5.24	4.30	-
I-LLM [33]	(8)	W6A6	per-channel	mixed	5.84	5.23	4.32	-
		W4A4	per-channel	mixed	9.10	7.99	7.24	-
ABQ-LLM [65]	(8)	W6A6	per-channel	per-token	5.81	5.21	-	-
		W4A4	per-channel	per-token	8.63	7.69	-	-

^{*} Results taken from [21], not the original paper.

[†] Results taken from [55], not the original paper.

[‡] Results taken from [70], not the original paper.

E. QUANTITATIVE ANALYSIS

As highlighted in Section I, the primary objectives of neural network quantization are to minimize model size and accelerate inference, while introducing as little error as possible. We evaluate quantized models against FP16 baselines, detailing compression ratios based on bit-widths—especially in the W4A4 configuration. We excluded direct latency measurements due to inconsistent reporting practices and hardware variability. However, theoretical throughput

implications can be inferred using the roofline model presented in Section II-B, which relates arithmetic intensity to performance bottlenecks. We primarily utilize WikiText-2 [79] perplexity, a widely reported metric, on the open-source LLaMA model family [80], [81], [82] to gauge the impact of quantization on model performance. However, we also include comparisons involving accuracy on the GSM8k dataset [83] and measurements conducted on OPT [84] and Mistral [85] models to provide a broader perspective. Additionally, we examine quantization time (defined as the duration required to compress a model from its original state) when applicable, as it is a critical factor in post-training quantization.

It is important to note that, while this analysis offers a broad comparison of quantization effects, readers are encouraged to refer to the original papers for detailed and nuanced results across various configurations. Unless explicitly stated otherwise, the reported results are derived directly from these original works.

1) EFFECTIVENESS OF CHANNEL-WISE SCALING

TABLE 3. Comparison of rotation techniques based on W4A4 WikiText-2 Perplexity for LLaMA-2 models. In the Table, RTN Stands for round-to-nearest, LWC stands for learnable weight clipping, and GPTQ refers to the weight quantization method introduced in [77].

Method	Rotation matrix	Quantization time [*]	Weight quantizer	Perplexity (↓)		
				7B	13B	70B
FP16 baseline	-	-	-	5.47	4.88	3.31
QuaRot [35]	Random Hadamard	5m	RTN [†]	8.56	6.10	4.14
			GPTQ [‡]	6.10	5.40	3.79
DuQuant [28]	Greedy search	4.5m	RTN	6.28	5.42	3.79
			LWC	6.08	5.33	3.76
SpinQuant [52]	CayleySGD	3.5h	RTN [†]	6.14	5.44	3.82
			GPTQ [‡]	5.96	5.24	3.70

^{*} Measured on LLaMA2-70B considering RTN weight quantization. QuaRot and DuQuant measurements were obtained on a single NVIDIA A100 GPU, while SpinQuant does not specify the hardware setup.

[†] Results taken from [57], not the original paper.

Table 2 compares the WikiText-2 perplexity of channel-wise scaling-based quantization techniques presented in Section IV-A1. The results are measured using the first LLaMA generation under different bit-widths and quantization granularities. Certain methods have been excluded from this comparison for the following reasons: Outlier Suppression [31] is designed for smaller-scale models and cannot effectively quantize LLaMA; QoQ [21] employs alternative equivalent transformations and is therefore included in later comparisons; and BiSup [72] is proposed as a supplementary method to enhance other techniques. The data shows that channel-wise scaling achieves negligible accuracy loss in the W8A8 setting compared to the baseline model. This holds even when the quantization parameters are shared across the entire activation tensor. Notably, when per-token activation quantization is applied, the perplexity difference between heuristically set and directly optimized scaling factors is minimal in the W6A6 and W4A8 settings. However, in the low-bit W4A4 setting, methods relying solely on heuristic scaling fail to achieve acceptable perplexity values. Even with the optimization of scaling factors, significant quantization

TABLE 4. Comparison of methods utilizing equivalent transformations based on WikiText-2 perplexity for LLaMA models.

Method	Bits	Gradient descent*	Equivalent transformations				Perplexity (↓)					
			Scaling	Shifting	Rotation	Affine	1-7B	1-13B	1-30B	2-7B	2-13B	3-8B
Baseline	FP16						5.68	5.09	4.10	5.47	4.88	6.14
SmoothQuant [25]	W6A6 [†]		✓				6.03	5.42	4.55	6.20	5.18	-
OS+ [60]	W6A6		✓	✓			5.76	5.22	4.30	-	-	-
LRQuant [70]	W6A6	✓	✓	✓			5.88	5.27	4.31	5.67	5.07	-
OmniQuant [55]	W6A6	✓	✓	✓			5.96	5.28	4.38	5.87	5.14	-
QoQ [21]	W4A8		✓		✓		5.89	5.25	4.28	5.70	5.08	6.76
DuQuant [28]	W4A4		✓		✓		6.18	5.47	4.55	6.08	5.33	8.06
RRS [63]	W4A4		✓		✓		-	-	-	-	5.45	8.42
SpinQuant [52]	W4A4	✓			✓		-	-	-	5.90	5.30	7.30
AffineQuant [53]	W4A4	✓	✓	✓	✓	✓	10.28	10.32	9.35	12.69	11.45	-
FlatQuant [57]	W4A4	✓	✓		✓	✓	-	-	-	5.78	5.11	6.90

* Indicates whether gradient descent is used to optimize the equivalent transformation parameters.

† Results taken from [55], not the original paper.

TABLE 5. Comparison of mixed-precision KV Cache quantization methods based on GSM8K accuracy and WikiText-2 Perplexity for LLaMA2-13B and mistral-7B models. In the Table, C and T Represent channel/token-wise quantization, respectively, and the symbol *g* denotes per-group quantization in the specified direction with the group size indicated.

Method	Avg bits	Granularity		Mixed-precision type			Accuracy (% , ↑)		Perplexity (↓)	
		K	V	Token	Channel	Element	LLaMA2	Mistral	LLaMA2	Mistral
Baseline	16	-	-				28.05	41.62	4.57	4.76
KIVI [51]	3.05	C-g32	T-g32	✓			23.65	37.30	-	-
GEAR [43]	4.42	C-g64	T-g64	✓		✓	30.27	43.14	-	-
ZipCache [41]	3.21	C	CT	✓			27.85	41.24	-	-
MiKV [34]	3.21*	T-g64	T-g64	✓			23.65	36.32	-	-
KVQuant [42]	2.34	C	T	✓		✓	-	-	4.78	5.14
Atom [20]	4.16 [†]	T-g64	T-g64		✓		-	-	4.71	4.82
SKVQ [38]	2.25	T-g128	T-g128	✓			-	-	4.87	-

* Results taken from [41], not the original paper.

† Results taken from [42], not the original paper.

errors persist, underscoring the inadequacy of channel-wise scaling alone in such constrained configurations.

2) PERFORMANCE OF ROTATION-BASED TECHNIQUES

Table 3 presents WikiText-2 perplexity results for the main rotation-based techniques used to quantize both weights and activations of the LLaMA2 family to 4 bits. While QuaRot requires careful handling of weight quantization to achieve acceptable results, DuQuant and SpinQuant show only minor improvements when this step is included. Furthermore, there is no notable difference in the perplexities obtained using the rotation matrices generated by DuQuant and SpinQuant. However, the greedy search algorithm employed in DuQuant introduces negligible quantization time overhead, whereas the gradient-based optimization technique used in SpinQuant significantly increases processing time.

3) COMPARISON OF EQUIVALENT TRANSFORMATION METHODS

The general comparison among equivalent transformation-based methods is provided in Table 4. While channel-wise scaling alone is insufficient for low-bit scenarios, it serves as a fundamental outlier mitigation strategy adopted by most techniques. Introducing additional channel-wise shifting significantly improves perplexity in the W6A6 setting. Moreover, rotation enables achieving comparable results even in the more aggressive W4A4 setting. For scenarios where sufficient resources are available, gradient-based techniques demonstrate the most favorable results in this category, particularly when incorporating learnable affine transformations. Notably, AffineQuant struggles to deliver satisfactory outcomes, likely due to its original design

TABLE 6. Comparison of block-based data formats for W4A4 quantization based on WikiText-2 perplexity (↓).

Method	Group size	OPT		LLaMA-2	
		6.7B	13B	7B	13B
FP16 baseline	-	10.86	10.13	5.47	4.88
BFP*	16	11.22	11.15	7.69	6.74
BiE [71]	16	10.93	10.39	7.00	6.16
MXFP4-FP8 [†]	32	13.71	12.09	6.49	5.69
AMXFP4-FP8 [45]	32	13.06	11.90	6.22	5.47

* Results taken from [71].

† Results taken from [45].

for weight-only quantization, which assumes unquantized activations during the optimization process.

4) IMPACT OF MIXED-PRECISION TECHNIQUES ON KV CACHE QUANTIZATION

Our analysis reveals that mixed-precision techniques are predominantly employed for KV cache quantization, and we therefore analyze their effectiveness from this perspective using WikiText-2 perplexity and GSM8k accuracy results, as summarized in Table 5. Comparing these methods is particularly challenging due to variations in the bit-widths used and inconsistencies in baseline results. From the table, it is evident that GEAR delivers superior performance despite achieving a lower compression rate for the KV cache. Additionally, ZipCache, leveraging channel-separable token-wise quantization, outperforms its counterparts at the same average bit-width. Moreover, KVQuant and SKVQ demonstrate perplexities comparable to Atom while achieving significantly higher compression ratios.

5) EVALUATION OF BLOCK-BASED DATATYPES FOR QUANTIZATION

Table 6 presents a comparison of WikiText-2 perplexities for OPT and LLaMA2 models quantized using various

TABLE 7. Overall comparison of evaluated papers with state-of-the-art QAT and weight-only quantization techniques based on WikiText-2 perplexity and total quantization time for the LLaMA family.

Category	Method	Quantization time (hours)	Bits			Perplexity (J)				
			W	A	KV	1-7B	1-65B	2-7B	2-70B	3-8B
Baseline	-	-	16	16	16	5.68	3.53	5.47	3.32	6.14
QAT	EfficientQAT [11]	41 (2-70B)	4	16	16	-	-	5.53	3.39	6.47
			3	16	16	-	-	5.81	3.61	7.09
	LR-QAT [86]	54 (2-7B)	4	8	8	5.85	-	5.67	-	-
Weight-only			4	4	4	8.47	-	8.46	-	-
	QuIP# [8]	100 (2-70B)	4	16	16	5.76	3.60	5.56	3.38	-
			3	16	16	5.98	3.78	5.79	3.56	-
	SqueezeLLM [10]	1.4 (1-65B)	4.27	16	16	5.77	3.63	5.57	3.39	-
			3.24	16	16	6.13	3.88	5.96	3.63	-
Weight-activation	ARB-LLM [13]	1.3 (2-7B)	1.08	16	16	14.03	6.56	16.44	6.16	27.42
	SpinQuant [52]	5.5 (2-70B)	4	8	8	-	-	5.70	3.50	6.50
			4	4	4	-	-	5.90	3.80	7.30
	FlatQuant [57]	6.15 (2-70B)	4	4	4	-	-	5.78	3.54	6.90
	Atom [20]	4 (1-65B)	4	8	4	5.95	-	5.80	3.53	7.04
			4	4	4	6.25	-	6.12	3.73	7.76
	QUTK [22]	2 (2-70B)	4	4	4	-	-	5.84	3.74	-
KV cache	AMXFP4 [45]	0	4	4	4	-	-	6.22	-	7.72
	WKVQuant [64]	3 (2-7B)	4	16	4	5.80	-	5.64	-	-
	KVQuant [42]	6 (1-65B)	16	16	2.34	6.05	3.72	-	-	-

^a Perplexity results taken from [21].

block-based datatypes. The bi-shared exponent approach introduced by BiE demonstrates superior performance compared to the simpler BFP format, as it effectively addresses the unique outlier distributions in activations, as evidenced by the results. Similarly, the asymmetric numerical representation employed in AMXFP4 surpasses the original microscaling format under identical settings. Notably, BFP and BiE perform better than MX-based formats on OPT models, whereas MX-based formats achieve lower perplexity on LLaMA2 models.

6) COMPREHENSIVE COMPARISON OF MAJOR QUANTIZATION TECHNIQUES

Finally, we compare several major techniques from the evaluated papers with state-of-the-art QAT and weight-only quantization techniques in Table 7. The table includes WikiText-2 perplexity results across various bit-widths for LLaMA models of different sizes and generations, along with the total quantization time for a specific model. We included ARB-LLM [13] to represent LLM binarization in our evaluation; however, it fails to achieve acceptable results compared to the FP16 baseline and significantly lags behind other evaluated techniques. EfficientQAT [11] and QuIP# [8] achieve results very close to the baseline when quantizing only weights, but their quantization processes require extensive GPU hours. In contrast, SqueezeLLM [10] achieves comparable results with slightly higher bit usage but significantly reduced quantization time. Despite this, these methods are suboptimal for compute-bound inference as discussed in Section II-B. LR-QAT [86] improves performance by leveraging integer arithmetic, though it requires 2.25 days to quantize a 7B model and falls short in achieving satisfactory results for 4-bit activation quantization, lagging behind other techniques. On the other hand, weight-activation quantization proves powerful by achieving results in the W4A4KV4 setting comparable to 3-bit weight-only quantization, while offering substantially higher peak performance and compressing the KV cache by a factor of 4. However, a noticeable perplexity

gap remains compared to 4-bit weight-only quantization. WKVQuant bridges part of this gap by reducing the KV cache size but struggles in compute-bound scenarios. Meanwhile, KVQuant aggressively compresses the cache while keeping weights at 16-bit precision, making it a practical choice for applications with long sequence lengths where the KV cache size surpasses the model size.

V. CONCLUSION

In this review, we conducted a comprehensive investigation into post-training activation quantization techniques for LLMs, specifically addressing the research questions posed in the introduction. Through a systematic selection process, we evaluated 52 articles, identified the main techniques employed by researchers (Question 1), analyzed them in detail, and provided various comparisons, including consideration of alternative methods such as weight-only quantization and QAT.

Our analysis revealed that activation outliers can be partially mitigated through mathematically equivalent transformations (Question 2). Among these, channel-wise scaling is the most frequently applied technique, smoothing activations in a token-wise manner and shifting quantization difficulty to weights, which generally exhibit a more quantization-friendly distribution. While this approach effectively addresses systematic outliers (first type) emerging in specific channels, it does not resolve channel asymmetry or massive outliers (second type). To address these, researchers have introduced channel-wise shifting to center activation distributions around zero, enabling symmetric quantization, which is more computationally efficient. Additionally, rotation techniques redistribute outlier values across channels, addressing the first type of outliers while also offering a solution for activation spikes. As a broader approach, some methods leverage learnable affine transformations. We detail various methods for obtaining equivalent parameters, such as scaling factors and rotation matrices, ranging from heuristic designs to gradient-based optimization, and report their

computational requirements and quantization performance. Despite these advancements, the composition of optimal equivalent parameters remains an open research question.

Another line of research focuses on modifying quantization parameters to mitigate outliers, rather than addressing activation distributions directly (Question 3). Techniques in this category include adjusting the number of bits used, reducing the size of groups sharing quantization parameters, or reshaping the quantization grid. Some methods allocate more bits to specific structures (e.g., channels and tokens) to reduce quantization error caused by outliers. Other approaches reduce quantization group sizes to narrow per-group distributions. Additionally, non-uniform quantization techniques employing specialized data types (e.g., BFP- and MX-based formats) have been proposed. While these methods achieve acceptable accuracy losses, they often introduce computational overhead during inference, highlighting the need for further algorithm-hardware co-design improvements.

Our quantitative analysis indicates that weight-only quantization remains superior to weight-activation techniques in terms of minimizing accuracy loss (Question 4). However, as discussed in Section II-B, activation quantization is essential for achieving optimal inference speed and energy efficiency, underscoring the importance of continued research in this area.

ACKNOWLEDGMENT

The authors would like to thank the Doctoral School of Applied Informatics and Applied Mathematics of Óbuda University for their valuable support.

REFERENCES

- [1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.
- [2] Z. G. Yang and T. Váradi, "Training experimental language models with low resources, for the Hungarian language," *Acta Polytechnica Hungarica*, vol. 20, no. 5, pp. 169–188, 2023.
- [3] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," 2023, *arXiv:2308.07633*.
- [4] W. Wang, W. Chen, Y. Luo, Y. Long, Z. Lin, L. Zhang, B. Lin, D. Cai, and X. He, "Model compression and efficient inference for large language models: A survey," 2024, *arXiv:2402.09748*.
- [5] A. Kuzmin, M. Nagel, M. V. Baalen, A. Behboodi, and T. Blankevoort, "Pruning vs quantization: Which is better?" presented at the NeurIPS, Nov. 2023.
- [6] H. Qin, X. Ma, and X. Zheng, "Accurate LoRAFinetuning quantization of LLMs via information retention," in *Proc. PMLR*, vol. 235, 2024, p. 41.
- [7] J. Chee, V. Kuleshov, Y. Cai, and C. D. Sa, "QuIP: 2-bit quantization of large language models with guarantees," presented at the NeurIPS, vol. 36, 2023.
- [8] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, "QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks," presented at the PMLR, vol. 235, 2024, p. 48 630–48 656.
- [9] T. Dettmers, R. Svirschevski, and V. Egiazarian, "SPQR: A sparse-quantized representation for near-lossless LLM weight compression," presented at the ICLR, 2024.
- [10] S. Kim, C. Hooper, and A. Gholami, "SqueezeLLM: Dense-and-sparse quantization," presented at the *Proc. PMLR*, vol. 235, 2024, pp. 23901–23923.
- [11] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K. Zhang, and P. Luo, "EfficientQAT: Efficient quantization-aware training for large language models," 2024, *arXiv:2407.11062*.
- [12] W. Huang, Y. Liu, and H. Qin, "BiLLM: Pushing the limit of post-training quantization for LLMs," in *Proc. PMLR*, vol. 235, 2024, pp. 20023–20042.
- [13] Z. Li, X. Yan, and T. Zhang, "ARB-LLM: Alternating refined binarizations for large language models," presented at the ICLR, Oct. 2024.
- [14] R. Gong, Y. Ding, Z. Wang, C. Lv, X. Zheng, J. Du, H. Qin, J. Guo, M. Magno, and X. Liu, "A survey of low-bit large language models: Basics, systems, and algorithms," 2024, *arXiv:2409.16694*.
- [15] Y. Wang, T. Yang, X. Liang, G. Wang, H. Lu, X. Zhe, Y. Li, and L. Weita, "Art and science of quantizing large-scale models: A comprehensive overview," 2024, *arXiv:2409.11650*.
- [16] M. J. Page, J. E. McKenzie, and P. M. Bossuyt, "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *Int. J. Surg.*, vol. 88, pp. 105–906, Apr. 2021.
- [17] B. Jacob, S. Kligys, and B. Chen, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, Jun. 2018, pp. 2704–2713.
- [18] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," 2021, *arXiv:2106.08295*.
- [19] G. Park, B. Park, and M. Kim, "LUT-GEMM: Quantized matrix multiplication based on LUTs for efficient inference in large-scale generative language models," presented at the ICLR, Oct. 2023.
- [20] Y. Zhao, C.-Y. Lin, and K. Zhu, "ATOM: Low-bit quantization for efficient and accurate LLM serving," in *Proc. MLSys*, 2024, pp. 196–209.
- [21] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "QServe: W4A8KV4 quantization and system co-design for efficient LLM serving," 2024, *arXiv:2405.04532*.
- [22] S. Ashkboos, I. Markov, E. Frantar, T. Zhong, X. Wang, J. Ren, T. Hoefler, and D. Alistarh, "QUICK: Towards end-to-end 4-bit inference on generative large language models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2024, pp. 3355–3371.
- [23] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [24] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," presented at the NeurIPS, vol. 35, 2022.
- [25] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. PMLR*, vol. 202, 2023, pp. 38087–38099.
- [26] J. Yang, H. Kim, and Y. Kim, "Mitigating quantization errors due to activation spikes in GLU-based LLMs," 2024, *arXiv:2405.14428*.
- [27] M. Chen, Y. Liu, J. Wang, Y. Bin, W. Shao, and P. Luo, "PrefixQuant: Eliminating outliers by prefixed tokens for large language models quantization," 2024, *arXiv:2410.05265*.
- [28] H. Lin, H. Xu, Y. Wu, J. Cui, Y. Zhang, L. Mou, L. Song, Z. Sun, and Y. Wei, "DuQuant: Distributing outliers via dual transformation makes stronger quantized LLMs," 2024, *arXiv:2406.01721*.
- [29] Q. Li, Y. Zhang, L. Li, P. Yao, B. Zhang, X. Chu, Y. Sun, L. Du, and Y. Xie, "FPTQ: Fine-grained post-training quantization for large language models," 2023, *arXiv:2308.15987*.
- [30] Z. Yuan, L. Niu, J. Liu, W. Liu, X. Wang, Y. Shang, G. Sun, Q. Wu, J. Wu, and B. Wu, "RPTQ: Reorder-based post-training quantization for large language models," 2023, *arXiv:2304.01089*.
- [31] X. Wei, Y. Zhang, and X. Zhang, "Outlier suppression: Pushing the limit of low-bit transformer language models," presented at the NeurIPS, vol. 35, 2022.
- [32] X. Ding, X. Liu, Z. Tu, Y. Zhang, W. Li, J. Hu, H. Chen, Y. Tang, Z. Xiong, B. Yin, and Y. Wang, "CBQ: Cross-block quantization for large language models," 2023, *arXiv:2312.07950*.
- [33] X. Hu, Y. Cheng, D. Yang, Z. Yuan, J. Yu, C. Xu, and S. Zhou, "I-LLM: Efficient integer-only inference for fully-quantized low-bit large language models," 2024, *arXiv:2405.17849*.
- [34] J. Y. Yang, B. Kim, J. Bae, B. Kwon, G. Park, E. Yang, S. J. Kwon, and D. Lee, "No token left behind: Reliable KV cache compression via importance-aware mixed precision quantization," 2024, *arXiv:2402.18096*.
- [35] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, "QuaRot: Outlier-free 4-bit inference in rotated LLMs," 2024, *arXiv:2404.00456*.

- [36] W. Byun, J. Woo, and S. Mukhopadhyay, "Hessian-aware KV cache quantization for LLMs," presented at the IEEE 67th Int. Midwest Symp. Circuits Syst. (MWSCAS), Aug. 2024.
- [37] S. Dong, W. Cheng, J. Qin, and W. Wang, "QAQ: Quality adaptive quantization for LLM KV cache," 2024, *arXiv:2403.04643*.
- [38] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. Zhang, and D. Lin, "SKVQ: Sliding-window key and value cache quantization for large language models," 2024, *arXiv:2405.06219*.
- [39] Y. Gao, J. Ou, L. Wang, Y. Xiao, X. Xiangzhiyuan, R. Dai, and J. Cheng, "Compensate quantization errors: Make weights hierarchical to compensate each other," in *Proc. Findings Assoc. Comput. Linguistics (NAACL)*, 2024, pp. 2711–2722.
- [40] Y. Gao, J. Ou, L. Wang, F. Shang, J. Wu, and J. Cheng, "Compensate quantization errors+: Quantized models are inquisitive learners," 2024, *arXiv:2407.15508*.
- [41] Y. He, L. Zhang, W. Wu, J. Liu, H. Zhou, and B. Zhuang, "ZipCache: Accurate and efficient KV cache quantization with salient token identification," 2024, *arXiv:2405.14256*.
- [42] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," 2024, *arXiv:2401.18079*.
- [43] H. Kang, Q. Zhang, S. Kundu, G. Jeong, Z. Liu, T. Krishna, and T. Zhao, "GEAR: An efficient KV cache compression recipe for near-lossless generative inference of LLM," 2024, *arXiv:2403.05527*.
- [44] J. Lee, M. Kim, S. Baek, S. Hwang, W. Sung, and J. Choi, "Enhancing computation efficiency in large language models through weight and activation quantization," presented at the Conf. Empirical Methods Natural Lang. Process., 2023.
- [45] J. Lee, J. Park, J. Kim, Y. Kim, J. Oh, J. Oh, and J. Choi, "AMXFP4: Taming activation outliers with asymmetric microscaling floating-point for 4-bit LLM inference," 2024, *arXiv:2411.09909*.
- [46] J. Liu, R. Gong, X. Wei, Z. Dong, J. Cai, and B. Zhuang, "QLLM: Accurate and efficient low-bitwidth quantization for large language models," presented at the ICLR, 2024.
- [47] L. Liu, H. Ren, L. Cheng, Z. Xu, Y. Pan, M. Wang, X. Li, Y. Han, and Y. Wang, "COMET: Towards partial W4A4KV4 LLMs serving," 2024, *arXiv:2410.12168*.
- [48] P. Liu, Z.-F. Gao, X. Zhao, Y. Ma, T. Wang, and J.-R. Wen, "Unlocking data-free low-bit quantization with matrix decomposition for KV cache compression," presented at the 62nd Annu. Meeting Assoc. Comput. Linguistics (Long Papers), 2024.
- [49] S.-Y. Liu, Z. Liu, X. Huang, P. Dong, and K.-T. Cheng, "LLM-FP4: 4-bit floating-point quantized transformers," presented at the Conf. Empirical Methods Natural Lang. Process., 2023.
- [50] W. Liu, X. Ma, P. Zhang, and Y. Wang, "CrossQuant: A post-training quantization method with smaller quantization kernel for precise large language model compression," 2024, *arXiv:2410.07505*.
- [51] Z. Liu, J. Yuan, and H. Jin, "KIVI: A tuningfree asymmetric 2bit quantization for KV cache," in *Proc. PMLR*, vol. 235, 2024, pp. 32332–32344.
- [52] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, "SpinQuant: LLM quantization with learned rotations," 2024, *arXiv:2405.16406*.
- [53] Y. Ma, H. Li, and X. Zheng, "AffineQuant: Affine transformation quantization for large language models," presented at the ICLR, 2024.
- [54] T. F. A. van der Ouderaa, M. L. Croci, A. Hilmkil, and J. Hensman, "Pyramid vector quantization for LLMs," 2024, *arXiv:2410.16926*.
- [55] W. Shao, M. Chen, and Z. Zhang, "OmniQuant: Omnidirectionally calibrated quantization for large language models," presented at the ICLR, 2024.
- [56] S. Son, W. Park, W. Han, K. Kim, and J. Lee, "Prefixing attention sinks can mitigate activation outliers for large language model quantization," 2024, *arXiv:2406.12016*.
- [57] Y. Sun, R. Liu, H. Bai, H. Bao, K. Zhao, Y. Li, J. Hu, X. Yu, L. Hou, C. Yuan, X. Jiang, W. Liu, and J. Yao, "FlatQuant: Flatness matters for LLM quantization," 2024, *arXiv:2410.09426*.
- [58] Y. Tan, H. Wang, C. Yan, and Y. Deng, "AlignedKV: Reducing memory access of KV-cache with precision-aligned quantization," 2024, *arXiv:2409.16546*.
- [59] Q. Tao, W. Yu, and J. Zhou, "AsymKV: Enabling 1-bit quantization of KV cache with layer-wise asymmetric quantization configurations," 2024, *arXiv:2410.13212*.
- [60] X. Wei, Y. Zhang, Y. Li, X. Zhang, R. Gong, J. Guo, and X. Liu, "Outlier Suppression+: Accurate quantization of large language models by equivalent and effective shifting and scaling," presented at the Conf. Empirical Methods Natural Lang. Process., 2023.
- [61] Z. Yao, X. Wu, C. Li, S. Youn, and Y. He, "Exploring post-training quantization in LLMs from comprehensive study to low rank compensation," presented at the *Proc. AAAI*, vol. 38, 2024, pp. 19377–19385.
- [62] Z. Yao, R. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers," presented at the NeurIPS, vol. 35, 2022.
- [63] K. Yi, Z. Liu, J. Zhang, C. Li, T. Zhang, J. Lin, and J. Zhou, "Rotated runtime smooth: Training-free activation smoother for accurate INT4 inference," 2024, *arXiv:2409.20361*.
- [64] Y. Yue, Z. Yuan, H. Duanmu, S. Zhou, J. Wu, and L. Nie, "WKVQuant: Quantizing weight and key/value cache for large language models gains more," 2024, *arXiv:2402.12065*.
- [65] C. Zeng, S. Liu, Y. Xie, H. Liu, X. Wang, M. Wei, S. Yang, F. Chen, and X. Mei, "ABQ-LLM: Arbitrary-bit quantized inference acceleration for large language models," 2024, *arXiv:2408.08554*.
- [66] C. Zhang, J. Cheng, I. Shumailov, G. Constantinides, and Y. Zhao, "Revisiting block-based quantisation: What is important for sub-8-bit LLM inference?" presented at the Conf. Empirical Methods Natural Lang. Process., 2023.
- [67] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava, "KV cache is 1 bit per channel: Efficient large language model inference with coupled quantization," 2024, *arXiv:2405.03917*.
- [68] Y. Zhang, F. Yang, S. Peng, F. Wang, and A. Pan, "FlattenQuant: Breaking through the inference computebound for large language models with per-tensor quantization," in *Proc. LREC-COLING*, 2024, pp. 7356–7365.
- [69] Y. Zhang, P. Zhang, M. Huang, J. Xiang, Y. Wang, C. Wang, Y. Zhang, L. Yu, C. Liu, and W. Lin, "QQQ: Quality quattuor-bit quantization for large language models," 2024, *arXiv:2406.09904*.
- [70] J. Zhao, M. Zhang, C. Zeng, M. Wang, X. Liu, and L. Nie, "LRQuant: Learnable and robust post-training quantization for large language models," presented at the 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers), 2024.
- [71] L. Zou, W. Zhao, S. Yin, C. Bai, Q. Sun, and B. Yu, "BiE: Bi-exponent block floating-point for large language models quantization," in *Proc. PMLR*, vol. 235, 2024, pp. 62978–62992.
- [72] M. Zou, R. Guo, S. Zhang, X. Zhang, and Z. Feng, "BiSup: Bidirectional quantization error suppression for large language models," 2024, *arXiv:2405.15346*.
- [73] E. J. Hu, Y. Shen, and P. Wallis, "LoRA: Low-rank adaptation of large language models," presented at the ICLR, Oct. 2021.
- [74] J. Li, F. Li, and S. Todorovic, "Efficient Riemannian optimization on the Stiefel manifold via the Cayley transform," presented at the ICLR, Sep. 2019.
- [75] Z. Zhang, Y. Sheng, and T. Zhou, "H2O: Heavyhitter Oracle for efficient generative inference of large language models," presented at the NeurIPS, Nov. 2023.
- [76] P. Liu, Z.-F. Gao, W. X. Zhao, Z.-Y. Xie, Z.-Y. Lu, and J.-R. Wen, "Enabling lightweight fine-tuning for pre-trained language model compression based on matrix product operators," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process. (Long Papers)*, 2021, pp. 5388–5398.
- [77] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "OPTQ: Accurate quantization for generative pretrained transformers," presented at the ICLR, 2023.
- [78] B. D. Rouhani, "Microscaling data formats for deep learning," 2023, *arXiv:2310.10537*.
- [79] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," presented at the ICLR, Feb. 2017.
- [80] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [81] H. Touvron, "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [82] A. Grattafiori, "The llama 3 herd of models," 2024, *arXiv:2407.21783*.
- [83] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," 2021, *arXiv:2110.14168*.

- [84] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.
- [85] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, "Mistral 7B," 2023, *arXiv:2310.06825*.
- [86] Y. Bondarenko, R. D. Chiaro, and M. Nagel, "Low-rank quantization-aware training for LLMs," 2024, *arXiv:2406.06385*.



PATRIK CZAKÓ (Student Member, IEEE) received the M.Sc. degree in computer science engineering from Budapest University of Technology and Economics, Budapest, Hungary, in 2022. He is currently pursuing the Ph.D. degree in information science and technology with Óbuda University, Budapest. His research interests include large language models, vision-language models, post-training model compression, and efficient fine-tuning approaches.



GÁBOR KERTÉSZ (Senior Member, IEEE) received the Ph.D. degree in information science and technology, in 2019.

He is currently an Associate Professor and the Vice-Dean for Research with the John von Neumann Faculty of Informatics, Óbuda University, Budapest, Hungary, and also a part-time Research Fellow with the HUN-REN SZTAKI (Institute for Computer Science and Control). He is the Leader of the Applied Machine Learning Research Group,

John von Neumann Faculty of Informatics. Main areas of his research was computer vision, parallel processing, and deep machine learning. His current research interests include metric learning and applied machine intelligence.

Dr. Kertész is the Vice-President of the John von Neumann Computer Society and the President of the IEEE Computational Intelligence Hungary Chapter.



SÁNDOR SZÉNÁSI (Member, IEEE) received the Ph.D. degree from the Doctoral School of Applied Informatics and Applied Mathematics, Óbuda University, Budapest, Hungary, in 2013. He is currently a Professor with the John von Neumann Faculty of Informatics, Óbuda University, and the Faculty of Economics and Informatics, J. Selye University. His research interests include (data) parallel algorithms, GPU programming, and medical image processing. He engages both in theoretical fundamentals and in algorithmic issues with respect to realization of practical requirements and given constraints.

...